

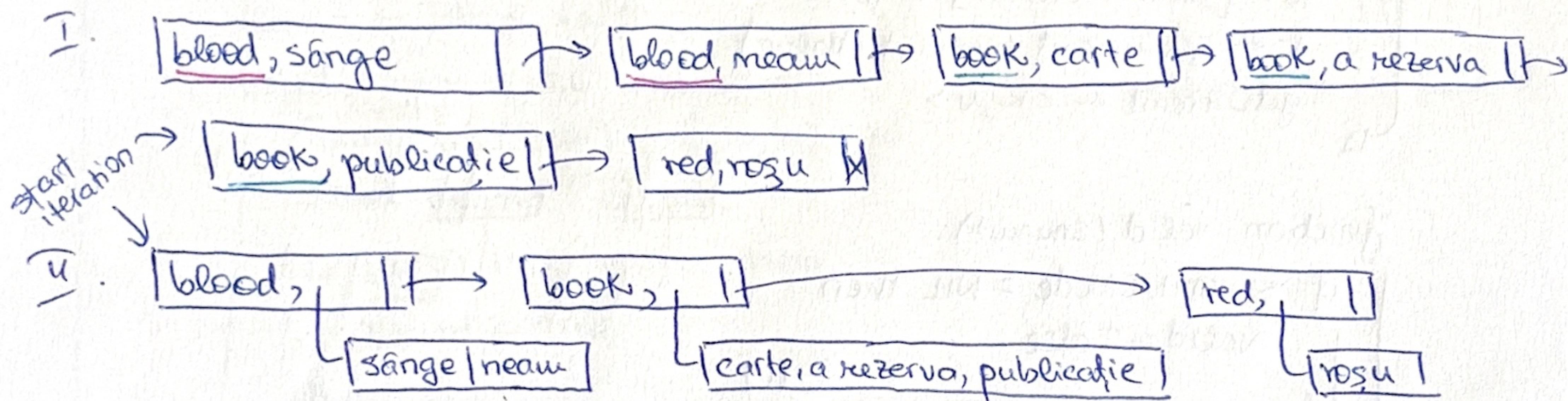
Seminar 4 - DSA

ADT Sorted Multiway (SMW)

22.04.2026
(official)

Implement ADT SMW - use a singly linked representation with dynamic allocation.

- book - carte, a rezerva, publicatie
- red - rosu
- blood - sange, neam



DS
DA
SU, DL

Implementation

init
valid
next
getCurrent
(no first!)
↓
no params.

ADT
Bag
List

Representation

TElem

Kv: TKey
VL: List

SMWNode

info: TElem
next: TSMWNode

SMW

head: TSMWNode
size: Integer
rel: Relation

SMWIterator

cNode: TSMWNode
eValue: ListIterator
Smm: SMW

subalgorithm init(smmmit, smm):

```
smmmit.smm ← smm
smmmit.cNode ← smm.head
if smm.head ≠ NIL then
  iterator([smm.head].info.vl, smmmit.eValue)
```

subalgorithm next(smmmit):

```
if not valid(smmmit)
  @throw exception
next(smmmit.eValue)
if not valid(smmmit.eValue)
```

because I
pointer to
a node

list = LL: O(n) | $\Theta(1)$

If list = DA: $\Theta(1)$ | $\Theta(1)$

eValue: position | iterator (SAFEST)

getPosition | getCurrent

list — indexed
— iterated

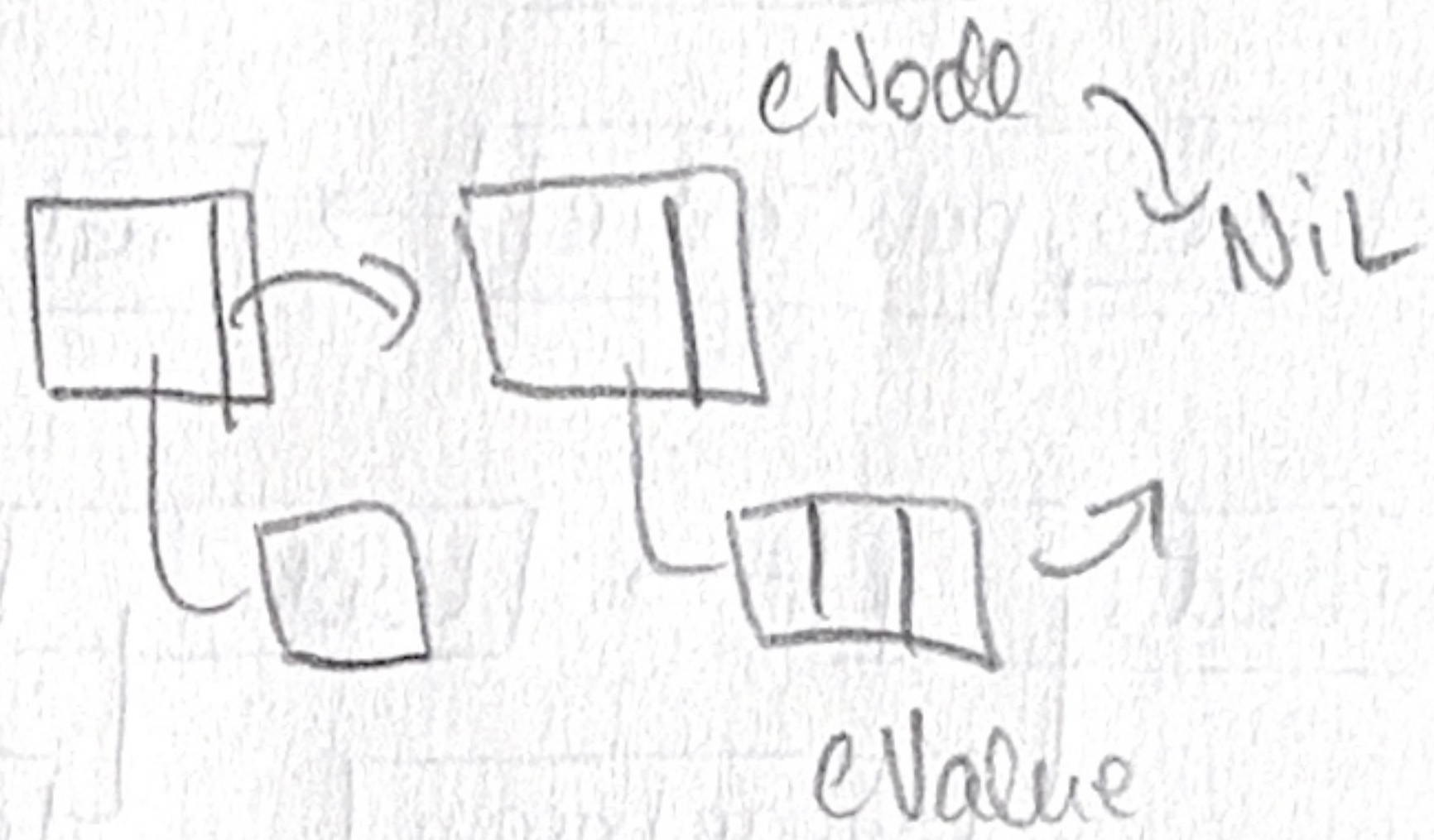
$\text{smmit.cNode} \leftarrow [\text{smmit.cNode}].\text{next}$ $\Theta(1)$
 if $\text{smmit.cNode} \neq \text{NIL}$ then
 $\text{iterater}([\text{smmit.cNode}].\text{info.vL}, \text{smmit.cValue})$

function $\text{getCurrent}(\text{smmit})$:

if not $\text{valid}(\text{smmit})$ then @ throw exception $\Theta(1)$
 $K \leftarrow [\text{smmit.cNode}].\text{info.K}$
 $v \leftarrow \text{getCurrent}(\text{smmit.cValue})$
 $\text{getCurrent} \leftarrow \langle K, v \rangle$

function $\text{valid}(\text{smmit})$:

if $\text{smmit.cNode} = \text{NIL}$ then
 $\text{valid} \leftarrow \text{False}$
 else
 $\text{valid} \leftarrow \text{True}$ $\Theta(1)$



add K, v pair

find node with K
 $\left\{ \begin{array}{l} \text{add } v \text{ to the value list} \\ \text{we need} \end{array} \right.$

remove (K, v)

find node with K

• remove v from value list

if value list is empty $\Rightarrow$ remove node.

$\Rightarrow$ we need the previous node.

search K

find node with K

~~subalgorithm $\text{searchNode}(\text{smmm}, K, KNode, \text{prevNode})$:~~

// K = book, KNode = "book", prevNode = "blood"

// K = blood, KNode = "blood", prevNode = Nil

// K = day, KNode = Nil, prevNode = book

// K = air, KNode = Nil, prevNode = Nil

Subalgorithm searchNode(smm, k, KNode, prevNode): → returns two pointers

aux ← smm.head

prev ← Nil

found ← ~~FALSE~~ false

while aux ≠ Nil and ~~smm.rel~~ ^{order matters?}
smm.rel([aux].info.k, k) and
not found execute

if [aux].info.k = k then

found ← true

else

prev ← aux

aux ← [aux].next

if found then:

KNode ← aux

prevNode ← prev

else

KNode ← Nil

prevNode ← prev (be aux passed the insertion position)

Subalgorithm add(smm, k, v):

searchNode(smm, k, KNode, prevNode) ← $O(m)$

if KNode ≠ Nil then

addToEnd([KNode].info.vL, v)

← $\Theta(1)$

else

newNode ← allocate()

[newNode].info.k ← k

init([newNode].info.vL)

addToEnd([newNode].info.vL, v)

if prevNode = Nil then // add it as the first elem

[newNode].next ← smm.head

smm.head ← newNode

1 5 9 11

should 1 be in front of ...?

$O(m)$

↳ m = nr. of distinct keys

$O(m)$

or

$O(\underline{m})$

↓

nr. of pairs

else

[new Node].next = [prev Node].next

[prev Node].next = new Node

Smm.size ← Smm.size + 1